

Claude Code Getting Started Guide (Windows Edition)

[Claude Code Getting Started Guide for Aspiring Web Developers \(Windows Edition\)](#)

[A Practical Guide for Building Prototype Websites on Windows](#)

[Table of Contents](#)

- [1. Setting Up Your Windows Machine](#)
- [2. Installing WSL2 \(Ubuntu on Windows\)](#)
- [3. Setting Up Git and GitHub](#)
- [4. Installing Windows Terminal](#)
- [5. Installing a Code Editor](#)
- [6. Installing Claude Code Prerequisites](#)
- [7. Installing Claude Code](#)
- [8. Installing Docker](#)
- [9. Creating Your First Project](#)
- [10. Using Claude Code Effectively](#)
- [11. Planning Mode and One-Shot Builds](#)
- [12. Recommended Free Tools](#)
- [13. Example Prompts to Try](#)
- [14. Working with Your Tech Partner](#)
- [15. Appendix: Native Windows Setup \(No WSL2\)](#)

[Quick Start Checklist](#)

[Getting Help](#)

Claude Code Getting Started Guide for Aspiring Web Developers (Windows Edition)

A Practical Guide for Building Prototype Websites on Windows

Table of Contents

1. [Setting Up Your Windows Machine](#)
 2. [Installing WSL2 \(Ubuntu on Windows\)](#)
 3. [Setting Up Git and GitHub](#)
 4. [Installing Windows Terminal](#)
 5. [Installing a Code Editor](#)
 6. [Installing Claude Code Prerequisites](#)
 7. [Installing Claude Code](#)
 8. [Installing Docker](#)
 9. [Creating Your First Project](#)
 10. [Using Claude Code Effectively](#)
 11. [Planning Mode and One-Shot Builds](#)
 12. [Recommended Free Tools](#)
 13. [Example Prompts to Try](#)
 14. [Working with Your Tech Partner](#)
 15. [Appendix: Native Windows Setup \(No WSL2\)](#)
-

1. Setting Up Your Windows Machine

Recommendation: Use WSL2 (Windows Subsystem for Linux)

Modern Windows machines are excellent development environments — once you turn on **WSL2**. WSL2 runs a real Ubuntu Linux kernel inside Windows, so you get the same `bash` shell, the same package manager (`apt`), and the same file paths (`~/projects`) that web developers use on Mac and Linux.

Why WSL2 is the right path for web development:

Factor	WSL2 on Windows	Native Windows (PowerShell)
Terminal/CLI	Real bash/zsh	PowerShell or Git Bash
Node.js/npm	Works seamlessly	Works, but path edge cases
Docker	Native WSL2 backend	Hyper-V backend (slower)
Git	Pre-installed in Ubuntu	Needs Git for Windows
Path issues	Rare	Common (spaces, backslashes)
Matches Mac guides/tutorials	Yes (almost line-for-line)	Often needs translation
Anthropic's recommendation	Yes	Supported but secondary

Hardware notes:

- Any Windows 10 (version 2004+) or Windows 11 machine will run WSL2.
- Aim for **16 GB RAM** if you can (8 GB works, but WSL2 + Docker + VS Code + a browser will feel tight).
- **256 GB SSD or larger**. A typical web project is under 500 MB, so you can store 50+ projects easily. Use GitHub as your backup/archive.
- WSL2 stores its Linux filesystem inside a virtual disk — by default on `C:\` . If your `C:` drive is small, see the WSL docs for moving the distro to another drive.

Tips to manage storage:

- Delete `node_modules` folders from inactive projects (run `npm install` to restore).
- Use `npx` to run tools without installing globally.
- Clean Docker images periodically: `docker system prune`.
- WSL2's disk image can grow over time — see Microsoft's docs for “compact WSL2 vhdx” when needed.

If you really cannot or do not want to use WSL2 (for example, your IT department blocks virtualization), see [Section 15: Native Windows Setup](#). The rest of the main guide assumes WSL2.

2. Installing WSL2 (Ubuntu on Windows)

WSL2 (Windows Subsystem for Linux, version 2) is Microsoft's official way to run Linux on Windows. Once installed, you'll have an Ubuntu terminal that behaves like a Mac or Linux development machine.

Step 1: Install WSL2 with one command

1. Click the **Start** menu, type “**PowerShell**”, right-click it, and choose “**Run as administrator**”.
2. In the PowerShell window, run:

```
wsl --install
```

This single command will:

1. Enable the Windows features WSL needs.
2. Download and install the WSL2 Linux kernel.
3. Install Ubuntu (the default Linux distribution).
4. Prompt you to **restart your computer**.

After the restart, Ubuntu will finish setting itself up and open a terminal window. It will ask you to create a **UNIX username** and **password**:

- Pick a short, lowercase username (e.g., `dave`). It doesn't have to match your Windows username.
- Pick a password you'll remember — you'll occasionally need it for `sudo` commands.
- The password will look like nothing is being typed when you enter it. That's normal in Linux.

About `sudo` : In Linux, `sudo` runs a command as the system administrator. You'll need it a few times during setup, mostly for `sudo apt install ...`. Don't make a habit of running random scripts with `sudo` — if you accidentally run malicious code as administrator, it can damage the entire Linux environment.

Step 2: Verify WSL2 is working

Open a new PowerShell window (regular, not admin) and run:

```
wsl --status
```

You should see something like:

```
Default Distribution: Ubuntu
Default Version: 2
```

If it says **Version: 1**, run this to upgrade:

```
wsl --set-default-version 2
wsl --set-version Ubuntu 2
```

Step 3: Update Ubuntu's package list

Open Ubuntu (search "Ubuntu" in the Start menu, or just type `wsl` in PowerShell) and run:

```
sudo apt update
sudo apt upgrade -y
```

This updates the list of available packages and upgrades anything that came pre-installed. It's the WSL/Ubuntu equivalent of `brew update && brew upgrade` on a Mac.

Step 4: Learn the basics of `apt`

`apt` is Ubuntu's package manager. It's the equivalent of Homebrew on Mac.

```
# Install a program
sudo apt install <package-name>

# Update the list of available packages
sudo apt update

# Upgrade everything you have installed
sudo apt upgrade

# Remove a program
sudo apt remove <package-name>

# Search for a package
apt search <keyword>
```

You'll use `apt` for system tools (Git, curl, build tools). For Node.js and Claude Code we'll use a different tool (`nvm`), and for GUI Windows apps we'll use **winget** in PowerShell — covered below.

Step 5: Understand your two filesystems

WSL2 gives you two filesystems, and it matters which one you use:

Where	Path in Ubuntu	Path in Windows	When to use
Linux home	<code>~</code> or <code>/home/you</code>	<code>\\wsl\$</code> <code>\Ubuntu\home\you</code>	Your projects go here. Fast.

Where	Path in Ubuntu	Path in Windows	When to use
Windows drive	<code>/mnt/c/Users/You</code>	<code>C:\Users\You</code>	Only when you need to hand a file to a Windows-only program.

Rule of thumb: Keep all your code under `~/projects` in Ubuntu. Don't put projects under `/mnt/c/...` — WSL2 is dramatically slower when working across the boundary, and `npm install` can take 10x longer.

You can still open WSL files from Windows File Explorer by typing `\\wsl$\Ubuntu\home\<your-username>` in the address bar.

3. Setting Up Git and GitHub

Git tracks your code changes. GitHub stores your code online so your tech partner can access it.

Step 1: Install Git in Ubuntu

Ubuntu usually ships with Git, but it's worth installing the latest:

```
sudo apt install git -y
```

Verify installation:

```
git --version
```

Step 2: Create a GitHub account

1. Go to github.com.
2. Click “**Sign up**”.
3. Use your student email if you have one (students get free GitHub Pro).
4. Verify your email.

Step 3: Configure Git with your identity

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Step 4: Set up SSH keys (so you don't type passwords constantly)

```
# Generate a new SSH key
ssh-keygen -t ed25519 -C "your-email@example.com"

# Press Enter to accept default location
# Enter a passphrase (or leave blank for no passphrase)

# Start the SSH agent
eval "$(ssh-agent -s)"

# Add your key to the agent
ssh-add ~/.ssh/id_ed25519

# Copy your public key to the Windows clipboard
# (clip.exe is a Windows tool – WSL2 lets you call it from Ubuntu!)
cat ~/.ssh/id_ed25519.pub | clip.exe
```

macOS equivalent: On Mac the command is `pbcopy < ~/.ssh/id_ed25519.pub`. In WSL2 we pipe to `clip.exe` — a nice example of how WSL2 lets Linux and Windows tools talk to each other.

Now add the key to GitHub:

1. Go to GitHub → **Settings** → **SSH and GPG keys**.
2. Click “**New SSH key**”.
3. Paste your key (Ctrl+V) and save.

Step 5: Test your connection

```
ssh -T git@github.com
```

You should see: “**Hi username! You’ve successfully authenticated...**”

The first time, it will ask you to confirm GitHub’s fingerprint — type `yes` and press Enter.

4. Installing Windows Terminal

The default “Ubuntu” window works, but **Windows Terminal** is much better. It supports tabs, splits, and lets you open PowerShell, Ubuntu, and Command Prompt side-by-side.

Install Windows Terminal

Windows 11 comes with Windows Terminal pre-installed. If you’re on Windows 10 or want the latest version, install it from the Microsoft Store:

1. Open the **Microsoft Store**.
2. Search for “**Windows Terminal**”.
3. Click **Install**.

Or install it via `winget` from PowerShell:

```
winget install --id Microsoft.WindowsTerminal -e
```

Make Ubuntu the default profile

1. Open **Windows Terminal**.
2. Click the dropdown arrow (v) in the title bar → **Settings**.
3. Under **Startup** → **Default profile**, choose **Ubuntu**.
4. Save.

Now every new Terminal window opens straight into Ubuntu.

Quick Windows Terminal setup

1. **Settings** → **Appearance** → **Theme**: Dark.
2. **Settings** → **Profiles** → **Ubuntu** → **Appearance** → **Color scheme**: Try “One Half Dark” or “Solarized Dark”.
3. **Settings** → **Profiles** → **Ubuntu** → **Appearance** → **Font face**: Cascadia Mono or “Cascadia Code”, size 14.

Useful Windows Terminal shortcuts

- **New tab**: Ctrl + Shift + T
 - **Split pane vertically**: Alt + Shift + +
 - **Split pane horizontally**: Alt + Shift + -
 - **Move between panes**: Alt + arrow keys
 - **Search**: Ctrl + Shift + F
 - **Copy / Paste**: Ctrl + Shift + C / Ctrl + Shift + V
-

5. Installing a Code Editor

While Claude Code works in the terminal, you’ll want an editor to view and occasionally tweak files.

Recommendation: Visual Studio Code (VS Code)

It’s free, visual, beginner-friendly, has great Git integration, and — most importantly — has a first-class **WSL extension** that lets it edit Linux files seamlessly.

Install VS Code (on Windows, not in WSL)

VS Code itself runs on Windows. Don’t install it inside Ubuntu — let the Windows version connect to WSL.

Install via `winget` from PowerShell:

```
winget install --id Microsoft.VisualStudioCode -e
```

Or download from code.visualstudio.com and run the installer. During install, check “**Add to PATH**” and “**Register Code as an editor for supported file types**”.

Install the WSL extension

1. Open VS Code.
2. Click the **Extensions** icon (four squares) on the left.
3. Search for “**WSL**” (publisher: Microsoft).
4. Click **Install**.

Essential extensions for web development

Also install:

1. **Prettier** — Auto-formats your code.
2. **Live Server** — Preview websites instantly.
3. **GitLens** — See who changed what.
4. **Auto Rename Tag** — Rename HTML tags in pairs.

Open VS Code from your Ubuntu terminal

In Ubuntu, navigate to any folder and run:

```
code .
```

The first time, VS Code will install a small server inside WSL automatically. After that, `code .` will pop open a VS Code window connected to your Linux environment — you’ll see “**WSL: Ubuntu**” in the bottom-left corner. That means you’re editing Linux files with full speed and no path translation.

6. Installing Claude Code Prerequisites

Claude Code requires Node.js version 18 or higher.

Install Node.js using nvm (Node Version Manager)

`nvm` lets you easily switch Node versions. We install it inside Ubuntu using its official installer:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh  
| bash
```

Close and reopen your terminal (or run `source ~/.bashrc`) so nvm loads. Then:

```
# Verify nvm is working  
nvm --version  
  
# Install the latest LTS (Long Term Support) Node.js  
nvm install --lts  
  
# Make it the default  
nvm use --lts  
nvm alias default 'lts/*'  
  
# Verify Node.js installation  
node --version    # Should show v20.x.x or higher  
npm --version     # Should show 10.x.x or higher
```

Heads-up on “nvm-windows”: There’s a separate tool called `nvm-windows` that runs in PowerShell. **Don’t use it inside WSL.** Inside WSL2/Ubuntu, you want the real Linux `nvm` shown above. Only use `nvm-windows` if you’re doing the native-Windows path in Section 15.

7. Installing Claude Code

Step 1: Install Claude Code globally (inside Ubuntu)

```
npm install -g @anthropic-ai/claude-code
```

If you get a permission error, **don't** prefix it with `sudo` — that's a sign `nvm` wasn't loaded. Close and reopen your terminal and try again.

Step 2: Authenticate

```
claude
```

The first time you run this, it will:

1. Print a URL (and try to open your default Windows browser — WSL2 handles this automatically).
2. Ask you to log in to your Anthropic account (create one at console.anthropic.com if needed).
3. Authorize Claude Code.

If the browser doesn't open automatically, copy the URL from the terminal and paste it into your Windows browser manually. Then paste the auth code back into the terminal.

Step 3: Verify it works

```
claude --version
```

Important: API usage costs

Claude Code uses the Anthropic API, which has usage costs. As a student building prototypes:

- Start with the free tier if available.
- Monitor your usage at console.anthropic.com.

- Consider setting spending limits.

8. Installing Docker

Docker lets you run databases, servers, and other infrastructure in isolated “containers.” Claude Code can create and manage Docker containers for you.

Install Docker Desktop (on Windows)

On Windows, Docker installs on the Windows side and connects to WSL2 for performance. **Don’t try to install Docker directly inside Ubuntu** — Docker Desktop’s WSL2 integration is the right path.

Install via `winget` from PowerShell:

```
winget install --id Docker.DockerDesktop -e
```

Or download Docker Desktop from docker.com/products/docker-desktop.

After installation:

1. Launch **Docker Desktop**.
2. Accept the license.
3. Open Docker Desktop → **Settings** (gear icon) → **General**.
 - Check “**Use the WSL 2 based engine.**”
4. Settings → **Resources** → **WSL Integration**.
 - Toggle on “**Enable integration with my default WSL distro**”.
 - Also toggle on the **Ubuntu** entry below.
5. Click **Apply & Restart**.

Verify Docker is running from inside Ubuntu

In your Ubuntu terminal:

```
docker --version
docker ps    # Should show an empty list (no containers yet)
```

If `docker ps` errors with “permission denied” or “cannot connect to the Docker daemon,” go back to Docker Desktop’s **WSL Integration** settings and make sure Ubuntu is enabled, then close and reopen Ubuntu.

Keep Docker running

When working with Claude Code on projects that need databases:

- **Always start Docker Desktop first** before running Claude Code.
- Claude Code will automatically create containers when needed.
- Docker Desktop must be running, or database commands will fail.
- You can have Docker Desktop start automatically with Windows (Settings → General → “Start Docker Desktop when you sign in to your computer”).

Common Docker commands

```
# See running containers
```

```
docker ps
```

```
# See all containers (including stopped)
```

```
docker ps -a
```

```
# Stop all running containers
```

```
docker stop $(docker ps -q)
```

```
# Clean up unused resources (reclaim disk space)
```

```
docker system prune
```

```
# Remove all stopped containers and unused images (more aggressive)
```

```
docker system prune -a
```

Example: Claude Code with Docker

When you tell Claude Code something like “create a web app with a PostgreSQL database,” it will:

1. Create a `docker-compose.yml` file.
2. Run `docker compose up` to start the database.
3. Configure your app to connect to it.

You just need Docker Desktop running — Claude Code handles the rest.

9. Creating Your First Project

Create a projects directory

In Ubuntu (Windows Terminal → Ubuntu tab):

```
# Create a folder for all your projects in your Linux home
mkdir -p ~/projects

# The tilde ~ means your home directory (/home/<your-username>).
# You can always get there by typing: cd

# Navigate to it
cd ~/projects

# Verify the full path of the present working directory
pwd
```

You should see something like `/home/dave/projects` . **Keep your code here, not under `/mnt/c/...`** . WSL2 is dramatically faster on its native filesystem.

Create a new project

```
# Create a project folder
mkdir my-first-website

# Change directory into it
cd my-first-website

# Initialize Git
git init

# Create a basic README
echo "# My First Website" > README.md

# Make your first commit
git add .
git commit -m "Initial commit"
```

Connect to GitHub

1. Go to GitHub and click **"New repository"**.
2. Name it `my-first-website`.
3. Keep it **Private** (good practice for client work).
4. **Don't** initialize with a README (you already have one).
5. Click **Create repository**.

GitHub will show you commands. Run the "push an existing repository" ones:

```
git remote add origin git@github.com:YOUR-USERNAME/my-first-website.git
git branch -M main
git push -u origin main
```

Start Claude Code in your project

```
# Make sure you're in the project directory
cd ~/projects/my-first-website

# Start Claude Code
claude
```

10. Using Claude Code Effectively

Basic interaction

Once Claude Code is running, you type natural language requests:

```
> Create a simple landing page for a coffee shop called "Boiler Brew"
```

Claude Code will:

- 1. Understand your request.
- 2. Create the necessary files.
- 3. Show you what it's doing.
- 4. Ask questions if needed.

Essential commands

Command	What it does
/help	Show available commands
/clear	Clear conversation history
/cost	Show API usage costs
Ctrl+C	Cancel current operation
Ctrl+D or /exit	Exit Claude Code

Command	What it does
Shift+Enter	Next line of prompt entry
Enter	Accept prompt and begin processing

Windows Terminal note: Default copy/paste is **Ctrl+Shift+C** / **Ctrl+Shift+V** (not Ctrl+C / Ctrl+V — **Ctrl+C** cancels the running command, just like on Mac/Linux).

Tips for good prompts

Be specific about what you want:

Bad: "Make a website"

Good: "Create a single-page portfolio website for a photographer with a header,
gallery section showing 6 images in a grid, an about section, and a contact form. Use modern CSS with a dark theme."

Mention technologies if you have preferences:

"Create a landing page using HTML, Tailwind CSS, and vanilla JavaScript. No frameworks needed."

Describe the visual style:

"Use a minimalist design with lots of white space, a sans-serif font like Inter, and a blue (#0066CC) accent color."

11. Planning Mode and One-Shot Builds

The Power of Planning

Good planning leads to cleaner builds. Claude Code has a **Planning Mode** that helps you think through the project before writing code.

It takes time up front to think through and design what you want to have Claude build for you.

This up-front time investment pays dividends, however. Every 10 minutes invested in forethought and planning prior to the build will save hours of rework, testing, and tedious debugging later.

Using Planning Mode

Start your session by asking Claude Code to plan:

```
> I want to build a portfolio website for a freelance graphic designer.
Before we build anything, let's create a detailed plan. Please enter
planning mode and help me think through:
- What pages we need
- What components each page should have
- What the file structure should look like
- What technologies to use
Save the plan to CLAUDE.md when we're done.
```

Claude Code will:

1. Enter planning mode.
2. Explore options with you.
3. Ask clarifying questions.
4. Create a comprehensive plan.
5. Save it to `CLAUDE.md`.

The CLAUDE.md file

This is your project's "memory." It contains:

- Project overview
- Technical decisions
- File structure
- Implementation plan
- Notes for future sessions

Always create this file at the start of a project. Claude Code reads it automatically when you start a session in that folder.

Example CLAUDE.md structure

Project: Designer Portfolio

Overview

Portfolio website for Jane Doe, freelance graphic designer.

Tech Stack

- HTML5
- Tailwind CSS (via CDN)
- Vanilla JavaScript
- No backend needed (static site)

Pages

1. Home - Hero with name/tagline, featured work (3 items)
2. Portfolio - Grid gallery with category filters
3. About - Bio, skills, photo
4. Contact - Form (using Formspree for handling)

File Structure

```
/
├── index.html
├── portfolio.html
├── about.html
├── contact.html
├── css/
│   └── custom.css
├── js/
│   └── main.js
└── images/
    └── (placeholder images for now)
```

Design Notes

- Color palette: Navy (#1a365d), Gold (#d69e2e), White
- Font: Playfair Display for headings, Inter for body
- Mobile-first responsive design

Implementation Order

1. Set up file structure
2. Create shared header/footer
3. Build home page
4. Build portfolio page with filter
5. Build about page

6. Build contact page with form
7. Add responsive adjustments
8. Test all pages

One-Shot Build Strategy

After planning, aim for a **one-shot build** — giving Claude Code enough context to build everything correctly the first time.

```
> Now let's build this. Using the plan in CLAUDE.md, create the complete portfolio website. Build all pages, make sure navigation works between them, and include placeholder content. I want to be able to open index.html in a browser and have a fully functional site.
```

Why one-shot builds are better:

- Less back-and-forth means lower API costs.
- Fewer iterations mean fewer bugs.
- Complete context leads to more consistent code.
- You learn to communicate requirements clearly.

Tips for successful one-shot builds:

1. Spend 30% of your time on the plan.
2. Be very specific about requirements.
3. Include examples or references when possible.
4. Describe edge cases upfront.
5. Review the plan before building.

In the early stages of building and iteration, it may be better, faster, and more efficient to go back and update your plan, delete all of the previous work, and do another one-shot build from scratch with your updated plan.

Minor tweaks like changing colors, updating text, or even adding new pages can be done as modifications to the existing build; however, major revisions of existing pages and functionality may be better done as a new one-shot build. Remember to save your color, text, and page tweaks back to your plan, in case you decide to do another one-shot build later.

Managing Multiple Builds (Beginner Workflow)

When you're learning Claude Code, don't worry about Git right away. Use simple numbered directories for your early experiments:

```
~/projects/client-portfolio/
├─ CLAUDE.md           # Your master plan (keep at top level)
├─ builds/
│   ├─ v0.1/           # First attempt - learning the ropes
│   ├─ v0.2/           # Better prompts, improved design
│   └─ v0.3/           # Almost there...
└─ v1.0/               # Ready for client! Initialize Git here.
```

Why this works:

1. **Focus on one skill at a time** — Learn Claude Code first, Git later.
2. **Easy comparison** — Open two builds side-by-side in VS Code or File Explorer.
3. **No fear of breaking things** — Can't mess up a Git repo if there isn't one.
4. **Keep your experiments** — Failed attempts are learning opportunities, not clutter.

The workflow:

1. Create your project folder and write your plan in `CLAUDE.md`.
2. Set up and enter your first build directory:

```
# Create the builds folder and v0.1
mkdir -p ~/projects/client-portfolio/builds/v0.1

# Change into the build directory
cd ~/projects/client-portfolio/builds/v0.1

# Copy your plan here so Claude can read it
cp ../../CLAUDE.md .
```

3. Start Claude Code and do your one-shot build:

```
# Make sure you're in the build directory
```

```
pwd
```

```
# Should show: /home/yourname/projects/client-portfolio/builds/v0.1
```

```
# Start Claude Code
```

```
claude
```

Important: Claude Code creates files in your current directory. Always `cd` into the build folder before running `claude` .

4. Review the result. Update your master plan (`../../CLAUDE.md`) with lessons learned.

5. Create the next build and try again:

```
# Go back to project folder
```

```
cd ~/projects/client-portfolio
```

```
# Create next build directory
```

```
mkdir builds/v0.2
```

```
cd builds/v0.2
```

```
# Copy your updated plan
```

```
cp ../../CLAUDE.md .
```

```
# Run Claude Code again
```

```
claude
```

6. Repeat until you have something worth showing a client.

7. When you're happy, promote your best build to v1.0:

```
# Go back to project folder
cd ~/projects/client-portfolio

# Copy your best build to v1.0 (let's say v0.3 was the winner)
# NOTE: The cp command below creates the v1.0 directory automatically.
# Don't pre-create the v1.0 directory using mkdir, otherwise v0.3 will
  get created under
# the pre-existing v1.0 directory:
cp -r builds/v0.3 v1.0

# The CLAUDE.md should already be in there from when you built it
```

8. **Now** initialize Git in `v1.0/` and push to GitHub.

When to initialize Git (the v1.0 moment):

- The build is ready to show a client.
- You're handing off to your tech partner for deployment.
- You want to start tracking incremental changes (not wholesale rebuilds).

```
cd ~/projects/client-portfolio/v1.0
git init
git add .
git commit -m "Initial v1.0 build – ready for deployment"
```

Then connect to GitHub as shown in Section 9

After v1.0: Now Git makes sense. Use it to track changes, fixes, and enhancements. Your Git history will be clean and meaningful — showing intentional changes rather than your learning journey.

Pro tip: Keep your `builds/` folder around even after v1.0. You might want to reference an old approach, or your client might say “I liked that thing from the second version better.”

Cleaning Up Old Builds

Once you're confident you no longer need your experimental builds, you can delete them to free up disk space.

When it's safe to clean up:

- The client has approved the final version.
- The project has been deployed and is running smoothly.
- You're certain you won't need to reference old approaches.

Option 1: Use File Explorer (safest)

Open File Explorer and type `\\wsl$\Ubuntu\home\<your-username>\projects\client-portfolio` in the address bar. Drag the `builds` folder to the **Recycle Bin**. You can recover it if you change your mind (until you empty the bin).

Option 2: Delete from the terminal

```
# From the project folder
cd ~/projects/client-portfolio

# Delete all experimental builds at once
rm -rf builds/

# Or delete specific versions selectively
rm -rf builds/v0.1 builds/v0.2
```

Warning: `rm -rf` is permanent — there's no undo or Recycle Bin when deleting from inside WSL with `rm`. Double-check you're deleting the right folder before pressing Enter. If you're not 100% sure, use File Explorer instead.

12. Recommended Free Tools

Markdown Editor/Viewer

For editing CLAUDE.md and documentation:

MarkText (Free, cross-platform)

Install via `winget` from PowerShell:

```
winget install --id marktext.marktext -e
```

- Live preview as you type.
- Simple and lightweight.
- Works with files inside WSL via the `\\wsl$Ubuntu\...` path.

Alternative: VS Code has built-in Markdown preview (**Ctrl+Shift+V**).

Code Editor (Beyond VS Code)

If you find VS Code overwhelming:

Notepad++ (Free, Windows-only)

```
winget install --id Notepad++.Notepad++ -e
```

- Lightweight and fast.
- Good for quick file edits.

Zed (Free, now available on Windows)

- Very fast and minimal.
- Built-in AI features.
- Download from zed.dev.

Design/Prototyping

Figma (Free tier available)

- Browser-based, no install needed.
- Great for planning website layouts visually.
- Your designer clients might send you Figma files.

Browser for Development

Chrome or **Firefox Developer Edition**

- Built-in DevTools (right-click → **Inspect**).

- Essential for debugging CSS and JavaScript.

Install Chrome

```
winget install --id Google.Chrome -e
```

Or Firefox Developer Edition

```
winget install --id Mozilla.Firefox.DeveloperEdition -e
```

13. Example Prompts to Try

Starter project (try this first!)

Create a simple personal landing page with:

- A centered hero section with my name "Your Name" and tagline "Engineering Student & Web Developer"
- A brief about section
- Links to GitHub, LinkedIn, and email
- Use a clean, modern design with a dark theme
- Make it responsive for mobile

Business landing page

Create a landing page for a local tutoring business called "Boilermaker Tutoring":

- Hero section with headline, subheadline, and a "Book a Session" button
- Section listing 4 subjects offered (Math, Physics, Chemistry, CS)
- Testimonials section with 3 fake student reviews
- Contact section with address, phone, and email
- Use your university's colors (Old Gold and Black) as the color scheme
- Include smooth scroll navigation

Portfolio website

Build a 3-page portfolio website for a graphic designer:

- Home page with hero, featured work preview (3 items), and call to action
- Portfolio page with a grid of 9 project thumbnails with hover effects
- Contact page with a styled form

Use placeholder images from picsum.photos. Make the design minimal and typography-focused.

Interactive component

Create an interactive pricing calculator for web design services:

- Checkboxes for features (Landing Page \$500, Additional Pages \$200 each, Contact Form \$150, Gallery \$300, E-commerce \$1000)
- Slider for number of additional pages (0-10)
- Live total that updates as options change
- "Get Quote" button that shows a summary

Make it look professional with good spacing and animations.

With Docker (database project)

Create a simple bookmark manager web app with:

- Form to add a bookmark (URL, title, tags)
- List view of all bookmarks with search/filter
- Ability to delete bookmarks
- Use Node.js with Express for the backend
- PostgreSQL database for storage (use Docker)
- Simple frontend with vanilla HTML/CSS/JS

Make sure to include the `docker-compose.yml` file and instructions to run everything.

14. Working with Your Tech Partner

Sharing access via GitHub

Add your tech partner as a collaborator:

1. Go to your repository on GitHub.
2. **Settings** → **Collaborators**.
3. Add their GitHub username or email.

Branch workflow

Keep your main branch stable:

```
# Create a new branch for each feature
git checkout -b feature/new-landing-page

# Do your work with Claude Code...

# Commit your changes
git add .
git commit -m "Add new landing page design"

# Push the branch
git push -u origin feature/new-landing-page

# Create a Pull Request on GitHub for your partner to review
```

Communication through code

Add notes in CLAUDE.md for your partner:

Deployment Notes

- Site is static HTML/CSS/JS
- Can be deployed to Netlify, Vercel, or any static host
- No backend required
- Contact form uses Formspree (account: xyz@email.com)

For Tech Partner

- Database credentials are in .env (not committed)
- Docker setup: run ``docker compose up`` before starting
- See /docs folder for API documentation

.gitignore essentials

Create a `.gitignore` file to avoid committing unnecessary files:

Ask Claude Code to create it

> Create a `.gitignore` file appropriate for a Node.js web project

Or create it manually:

```
node_modules/  
.env  
.DS_Store  
*.log  
dist/  
.cache/  
Thumbs.db  
desktop.ini
```

The last two (`Thumbs.db` , `desktop.ini`) are Windows-specific hidden files that should never be committed.

15. Appendix: Native Windows Setup (No WSL2)

If you cannot use WSL2 — for example, your organization disables virtualization, or you specifically want to learn the Windows-native toolchain — here is the equivalent setup.

Everything in Sections 9–14 still applies, just substitute the install steps and paths below.

A1. Package manager: `winget`

Modern Windows (10 1809+ and all of Windows 11) ships with **winget**, Microsoft's package manager. It's the rough equivalent of Homebrew on Mac. Open PowerShell (no admin needed for most installs):

```
winget install --id Git.Git -e
winget install --id GitHub.cli -e
winget install --id Microsoft.WindowsTerminal -e
winget install --id Microsoft.VisualStudioCode -e
winget install --id Docker.DockerDesktop -e
winget install --id CoreyButler.NVMforWindows -e
```

A2. Use Git Bash (or PowerShell) as your shell

Git for Windows installs **Git Bash**, a bash-like shell. It understands most of the Mac/Linux commands in this guide (`ls` , `cd` , `mkdir` , `cp` , `rm` , `ssh-keygen`). Configure Windows Terminal so Git Bash is your default profile, the same way you'd set Ubuntu in the WSL path (Section 4).

Things that differ from the Mac/WSL guide:

- **Home directory** is `C:\Users\<You>` (Git Bash shows it as `/c/Users/<You>` and `~` still works).
- **Copy SSH key to clipboard:** in Git Bash, run `cat ~/.ssh/id_ed25519.pub | clip` (Windows has its own `clip` on PATH).
- **Path separators:** prefer forward slashes; Git Bash translates them.

A3. Node.js via `nvm-windows`

After installing `CoreyButler.NVMforWindows`, **close and reopen PowerShell** (so the installer's PATH changes take effect). Then:

```
nvm install lts
nvm use lts
node --version
npm --version
```

`nvm-windows` is a different program from the Linux `nvm`. Its commands look similar but it has fewer features. That's fine — you only need it to install one LTS version of Node.

A4. Install Claude Code

Same `npm` command as before — in **PowerShell** or **Git Bash**:

```
npm install -g @anthropic-ai/claude-code
claude --version
claude # first run will open the browser to authenticate
```

A5. Docker Desktop without WSL2

Docker Desktop on Windows without WSL2 falls back to the **Hyper-V** backend. It works, but it's slower and uses more RAM. Open Docker Desktop's settings and untick the WSL2 options. From PowerShell or Git Bash, `docker ps` should work once Docker Desktop is running.

A6. Project paths in native Windows

Use a folder under your Windows home for projects:

```
# In PowerShell
mkdir $HOME\projects
cd $HOME\projects
```

Or in Git Bash:

```
mkdir -p ~/projects  
cd ~/projects
```

~ in Git Bash is `C:\Users\<You>`, so everything else in this guide still works.

A7. When to switch back to WSL2

You'll probably want to revisit WSL2 when:

- You start using Linux-only tools (some CLIs only ship `.deb` packages).
- You're collaborating with someone on a Mac and want path/permission parity.
- `node_modules` installs feel slow.
- You're tired of fighting line-ending issues (`CRLF` vs `LF`) in Git.

WSL2 can be installed at any time later without redoing your projects — you can copy them from `C:\Users\<You>\projects` into `\\wsl$\Ubuntu\home\<you>\projects` once Ubuntu is set up.

Quick Start Checklist

☐

Install WSL2 (`wsl --install` in admin PowerShell, then reboot)

☐

Set up Ubuntu username and password

☐

Run `sudo apt update && sudo apt upgrade -y`

☐

Install Git in Ubuntu (`sudo apt install git -y`)

☐

Create GitHub account with student email

☐

Set up Git config and SSH keys (copy with `clip.exe`)

☐

Install Windows Terminal (`winget install Microsoft.WindowsTerminal`)

☐

Make Ubuntu the default profile in Windows Terminal

☐

Install VS Code on Windows (`winget install Microsoft.VisualStudioCode`)

☐

Install the WSL extension in VS Code

☐

Install nvm in Ubuntu (curl script)

☐

Install Node.js (`nvm install --lts`)

☐

Install Claude Code (`npm install -g @anthropic-ai/claude-code`)

☐

Create Anthropic account for API access

☐

Install Docker Desktop on Windows and enable WSL2 integration

☐

Create your ~/projects directory inside Ubuntu

☐

Create your first project folder

☐

Initialize Git and connect to GitHub

☐

Run Claude Code and try the starter prompt!

Getting Help

- **Claude Code issues:** Type `/help` or ask Claude Code itself.
- **Git problems:** Ask Claude Code “I’m having a git issue where...”
- **WSL2 issues:** `wsl --status` in PowerShell, or check Microsoft’s WSL docs.
- **General questions:** Claude Code can explain concepts too.

Remember: Claude Code is your pair programmer. Describe what you want clearly, plan before you build, and don’t be afraid to ask it questions. Good luck with your web prototyping business!

Windows companion to the Claude Code Getting Started Guide. Last updated: May 2026